

ProNeighbor - App Summary

One-page snapshot generated from repository evidence only.

What It Is

ProNeighbor is a gated-community marketplace web app that connects residents with verified local professionals and supports browsing, booking, messaging, wallet top-ups, and support workflows. It is a React + Firebase application with role-based user/admin experiences and PWA elements.

Who It's For

Primary users are residents and service professionals inside gated societies, with admins managing operations, moderation, and platform controls.

What It Does

- Supports auth flows: email/password, Google sign-in, password reset, email verification, and phone OTP validation.
- Provides protected user journeys for dashboards, pro browsing, pro details, booking flow, booking history, and booking detail pages.
- Includes in-app messaging with deterministic conversation IDs, attachments, real-time subscriptions, read receipts, and unread counts.
- Handles user profiles with residency proof upload/verification and safe public-profile mirroring.
- Offers a wallet/coin system with Razorpay checkout in Spark mode, plus Cloud Functions for server-verified Blaze mode.
- Delivers community features such as local feed posts/reactions/reports and recommendation helpers.
- Exposes extensive admin modules: users, societies, services, reviews, broadcasts, tickets, audit logs, wallet, bookings, and settings.

How It Works (Architecture)

- **Client app:** Vite + React + React Router + React Query bootstrapped in `src/main.tsx/src/App.tsx`.
- **Identity and roles:** Firebase Auth wrapped by `AuthProvider`; route gating via `ProtectedRoute` for user, verified-user, and admin access.
- **Data layer:** Firestore is the primary datastore accessed via service modules (profiles, bookings, services, messages, feed, societies, transactions, support).
- **Storage/media:** Firebase Storage is initialized; uploads are performed to Cloudinary for profile photos, residency proofs, and chat attachments.
- **Payments/coins:** Frontend wallet uses Razorpay SDK in Spark mode; optional Firebase Cloud Functions implement order creation, webhook verification, and ledger crediting.
- **Observability/UX:** Sentry init, `ErrorBoundary` wrapping, PWA splash/install components, and push messaging support checks.
- **Data flow:** Browser UI -> Auth/Firestore/Functions SDK calls -> Firebase services -> Firestore updates -> realtime listeners refresh UI.

How To Run (Minimal)

1. Install dependencies: `npm install`
2. Create env file: copy `.env.example` to `.env.local` and set Firebase values (plus optional Razorpay/Cloudinary keys used by wallet/uploads).
3. Start dev server: `npm run dev` (Vite runs on `http://localhost:5173` with strict port).

Not found in repo: single consolidated README for full setup/deployment checklist.

Evidence sampled from: `package.json`, `.env.example`, `vite.config.ts`, `src/App.tsx`, `src/main.tsx`, `src/firebase.ts`, `src/context/AuthContext.tsx`, `src/services/firestoreService.ts`, `src/services/razorpayService.ts`, `src/lib/collections.ts`, `functions/src/index.ts`, `src/pages/LandingPage.tsx`.